Algorytmy 2 *

Created	@December 4, 2024 9:07 PM
Class	Algorytmika

Podstawowe algorytmy matematyczne - Pojęcie rekurencji

Algorytm Euklidesa

Utworzony w IV wieku przed naszą erą

Służy on do wyznaczenia największego wspólnego dzielnika dwóch liczb

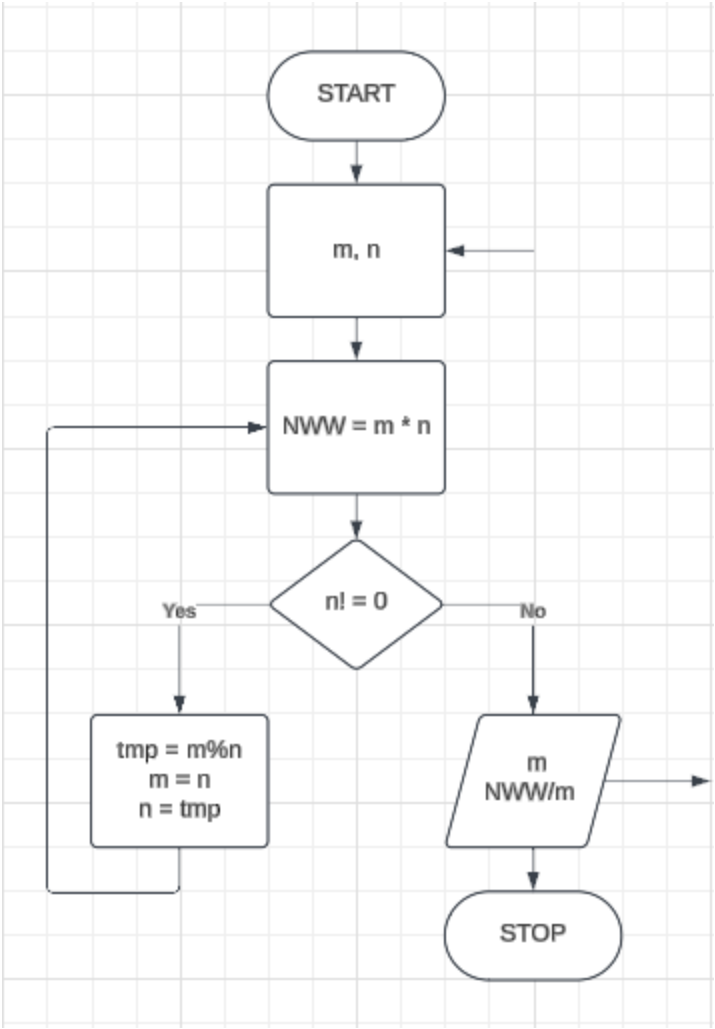
Jego zmodyfikowana wersja pozwoli nam na obliczenie najmniejszej wspólnej wielokrotności dwóch liczb

Pseudokod

1. m i n
2. NWW = m * n
3. n != 0
    a. tmp = m % n
    b. m = n
    c. n = tmp
4. NWW =/ m

NWW = a * b / NWD(a,b)

Schemat blokowy algorytmu Euklidesa



Program Algorytmu Euklidesa - NWW i NWD

```
#include <stdio.h>

void main(){
    int a,b,temp;
    printf("Podaj A: ");
    scanf("%d",&a);
    printf("Podaj B: ");
    scanf("%d",&b);
    int atemp = a;
    int btemp = b;
```

```
    while(b!=0)
    {
        temp = b;
        b = a%b;
        a = temp;
    }
    printf("NWD = %d", a);
    printf("\nNWW = %d", atemp*btemp/a);
}
```

Iteracja

Iterowanie to powtarzanie tej samej operacji w pętli z góry określoną liczbę razy lub aż do spełnienia określonego warunku

Rekurencja

Rekurencja działa podobnie do iteracji, lecz w tym przypadku, funkcja wywołuje samą siebie, z innymi parametrami aż do spełnienia pewnego warunku

Programy napisane rekurencyjnie, prawie zawszę mogą być przekształcone w formy iteracyjne

Przykłady rekurencji:

Informatyka:

- Wywołanie funkcji z jej wnętrza

Zastosowania w prawdziwym życiu:

- Matrioszka
- Udostępnianie ekranu
- Nagranie video

Cechy pozytywne:

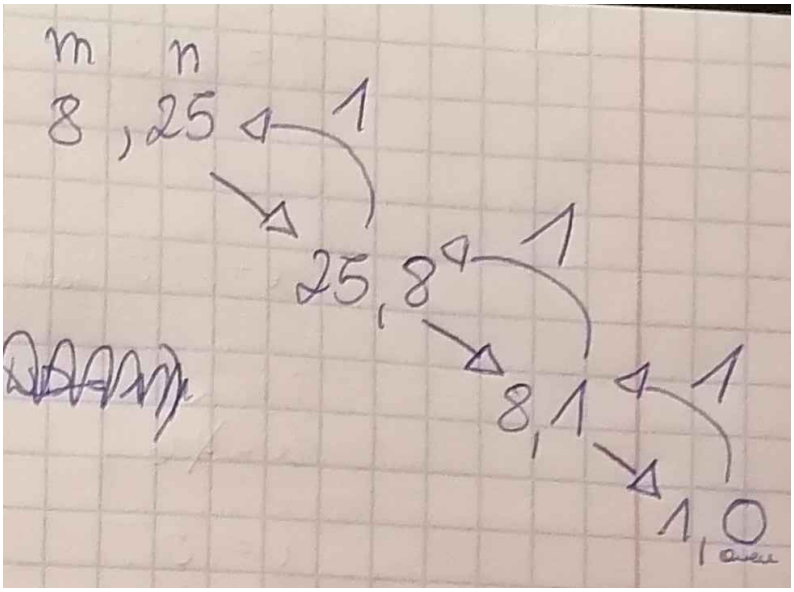
- Poprawia przejrzystość kodu
- Efektywna dla problemów rekurencyjnych
- Nie wykonujemy “na siłę”

Cechy negatywne:

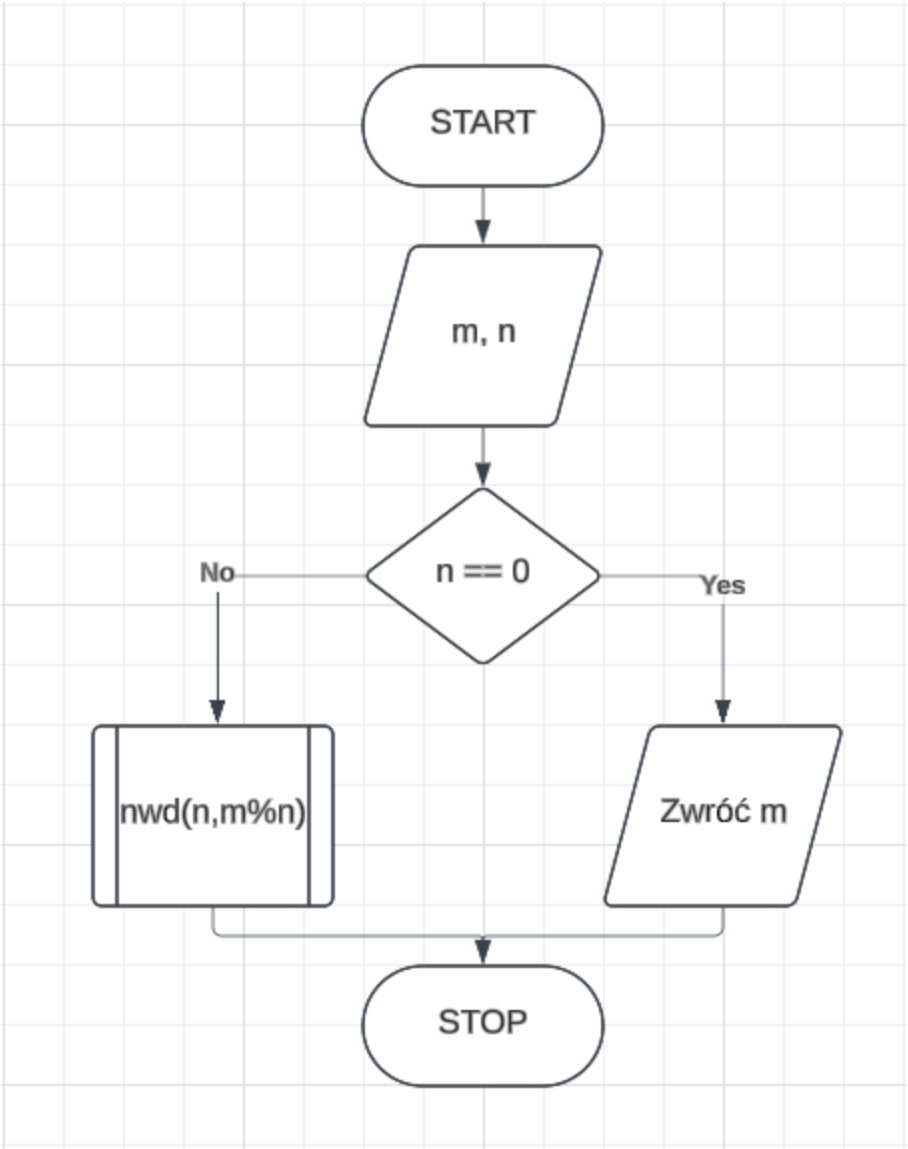
- Większe zużycie pamięci operacyjnej
- Możliwość przepełnienia stosu

Rekurencyjny algorytm Euklidesa

1. Podaj m oraz n
2. NWD(m,n) = NWD(n,m%n)
3. n == 0
    a. Zwróć m
    b. W przeciwnym wypadku:
        i. Wróć do kroku 2



Schemat blokowy algorytmu Euklidesa z rekurencją



Kod programu

```
int funk1(int a, int b)
{
    int tmp;
    while (b!=0)
    {
        tmp = a%b
        a = b;
        b = tmp;
    }
```

```
    return a;
}
int funk2(int a, int b)
{
    if(b==0)
    {
    return a;
    }
    else
    {
    return funk1(b,(a%b));
    }
}
```

Jest to algorytm naiwny - oznacza to, że działa poprawnie, lecz przeszukuje wszystkie elementy

Sito Eratostenesa

Powstał w III wieku przed naszą erą

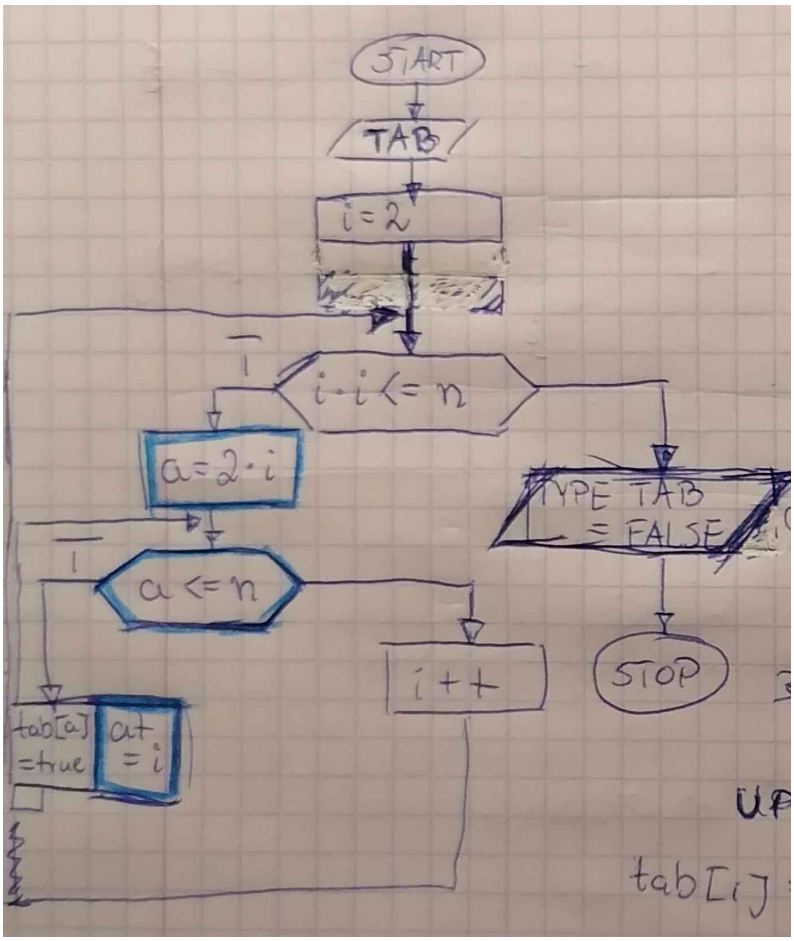
Jego zadaniem jest wyznaczenie wszystkich liczb pierwszych miejszych lub równych zadanej, czyli z przedziału [2,n]

Przesiewa zbiór w taki sposób, że pozostają jedynie liczby pierwsze

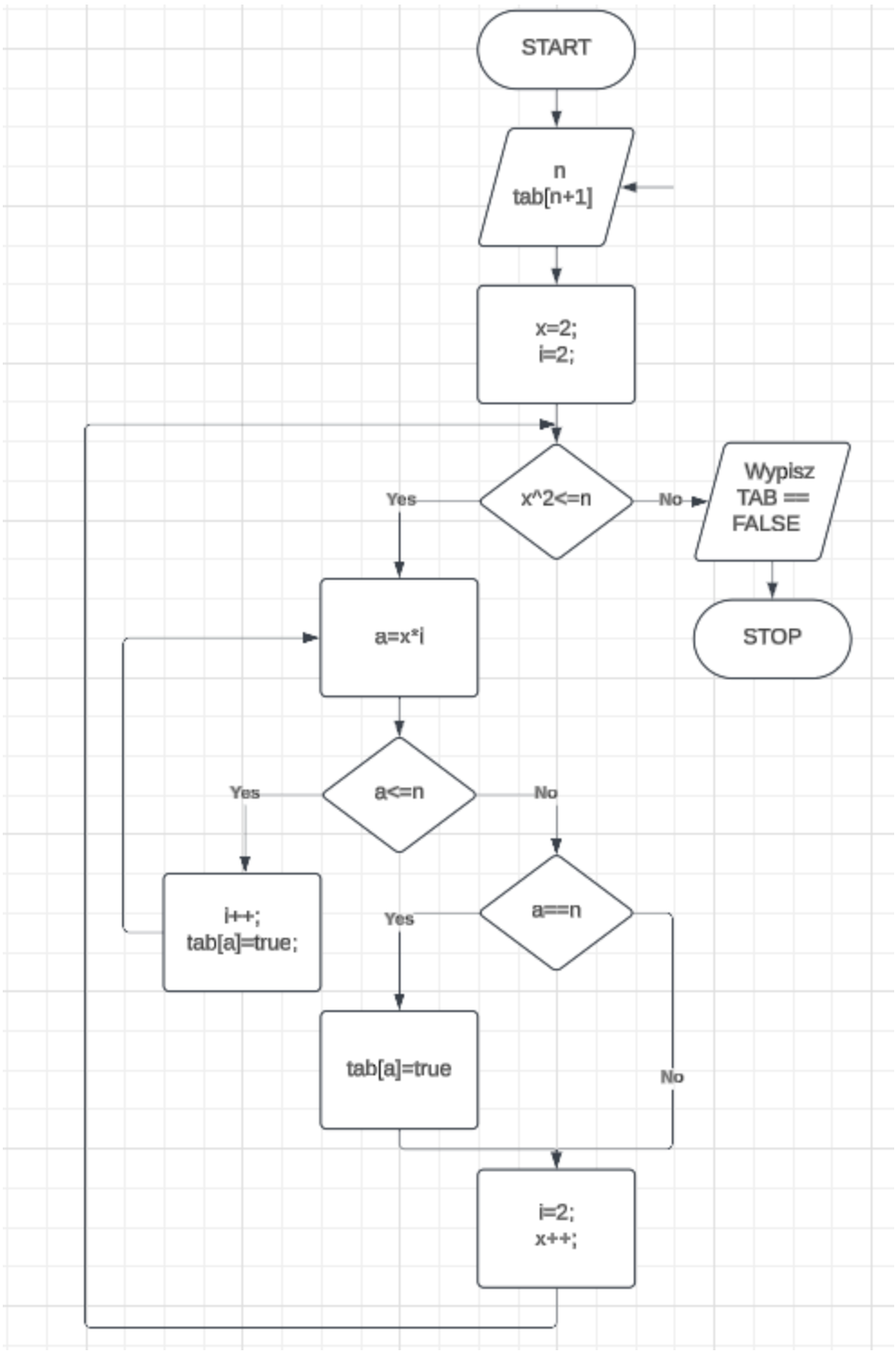
Pseudokod

1. Stwórz tablicę n+1 z wartościami false
2. Dla indeksów i = 2,3...,sqrt(n) wykonuj:
    a. a = 2*i, 3*i, 4*i.. aż do n
        i. Wykonaj tab[a] = true
3. Wypisz wszystkie indeksy od 2 do n, które mają wartość false

Schemat blokowy



Schemat blokowy - moja wersja



Kod programu

```
void funk1(int n)
{
```

```
    bool tab[n+1];
    for(int i = 0; i <= n; i++)
    {
        tab[i]=false;
    }
    for(int i = 2; i*i <= n; i++)
    {
        for(int a = 2*i; a<=n; a+=i)
        {
        tab[a]=true;
        }
    }
    for(int i=2; i<=n;i++)
    {
        if(!tab[i])
        {
            cout<<i<<" ";
        }
    }
```

Dodatek poprawiający kod

1. tab[i] == false, dla indeksów a=2*i, 3*i, 4*i... aż do n

    b. wykonaj: tab[a] = TRUE

Algorytm Fibbonaciego

Ciąg Fibbonaciego to ciąg liczb naturalnych, który zaczyna się od 0 i 1, a każda kolejna liczba jest sumą dwóch poprzednich

Ciąg ten został opisany przez włoskiego matematyka Leonardo Fibonnaciego w XII wieku

Algorytm ten pozwala nam obliczać kolejne elementy ciągu Fibbonaciego

Założenia algorytmu:

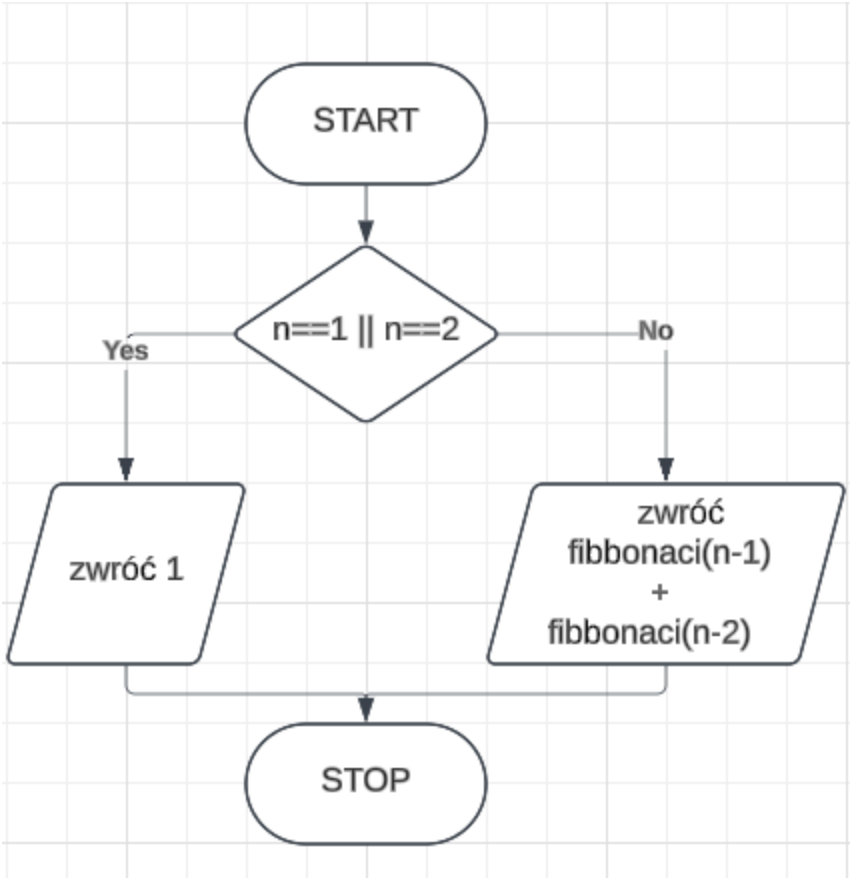
1. Procedura Fibbonaci

2. Weź numer elementu ciągu n
    a. Jeśli 0, zwróć 0
    b. Jeśli 1, zwróć 1
3. W przeciwnym wypadku zwróć Fibbonaci(n-1)+Fibbonaci(n-2)

Pseudokod

1. n (n≥2)
2. a = 0; b=1
3. dla i = 0,1,... n-2
    a. c = a+b
    b. a = b
    c. b = c
4. Wypisz b

Schemat blokowy algorytmu Fibbonaciego



Kod programu

```
// rekurencyjnie
int funk1(int n)
{
    if(n==0 || n==1)
    {
        return n;
    }
    else
    {
    return funk1(n-1)+funk1(n-2);
    }
    // iteracyjnie
    int funk2(int n)
    {
```

```
        int a = 0; int b = 1; int c;
        while(i<n-1)
        {
            c=a+b;
            a=b;
            b=c;
        } i++;
        return b;
    }
```

Pytania kontrolne

1. Do czego stosujemy algorytm Euklidesa?

- By znaleźć największy wspólny dzielnik dwóch liczb całkowitych, w jego zmodyfikowanej wersji, możemy znaleźć najmniejszą wspólną wielokrotność dwóch liczb

2. Iteracja vs rekurencja

- Iteracja to powtarzanie danego segmentu kodu aż do spełnienia danego wymogu, rekurencja to wywoływanie funkcji przez samą siebie z innymi danymi, aż do spełnienia danego wymogu

3. Zagrożenia wynikające z rekurencji

- Większe zużycie pamięci operacyjne, możliwość przepełnienia stosu

4. Co otrzymujemy z wyniku działania Sita Erastotenesa

- Otrzymujemy listę liczb pierwszych

5. Jak tworzy się kolejne elementy ciągu Fibonacciego

- Kolejne elementy ciągu Fibbonaciego powstają poprzez zsumowanie poprzednich dwóch elementów

6. Algorytm obliczania ciągu Fibonacciego jest rekurencyjne czy iteracyjne?

- Może być wykonany rekurencyjnie i iteracyjnie, lecz wersja rekurencyjna jest wydajniesza